# Automatic NPK-Based Fertilizer Spraying System

A Real-Time Soil Nutrient Monitoring & Precision Fertilization Project using Arduino UNO

## 1. Abstract

Fertilizer overuse and underuse are two of the most common and costly problems in modern farming and home gardening. Traditionally, farmers apply fertilizer on a fixed schedule or based on visual guesswork, without any real measurement of what the soil actually needs at that moment. This often leads to over-fertilization, which wastes money, damages plant roots through nutrient burn, and pollutes groundwater through chemical runoff, or under-fertilization, which leaves crops nutrient-starved and reduces yield and plant health.

This project solves that problem by building a closed-loop, sensor-driven fertilization system using an Arduino UNO as the central controller. At the heart of the system is a real RS485 Modbus RTU soil NPK sensor, an industrial-grade sensor capable of measuring actual Nitrogen, Phosphorus, and Potassium concentrations in the soil in milligrams per kilogram (mg/kg). Because this sensor communicates using the RS485 electrical standard rather than a simple analog voltage, a MAX485 transceiver module is used to bridge it to the Arduino's TTL serial pins, and the full Modbus RTU protocol - including CRC-16 error checking - is implemented in software to ensure every reading is genuine and uncorrupted.

The measured nutrient values are displayed continuously on a compact 128x32 pixel OLED screen, giving an immediate, readable view of soil health at any moment. When the system detects that any one of the three nutrients has dropped below a safe, pre-defined threshold, it automatically activates a relay-controlled fertilizer pump for a short, precise duration, delivering fertilizer only when and where it is actually needed. If nutrient levels are already sufficient, the pump remains completely off, conserving fertilizer and avoiding waste.

Beyond the core sensing-and-spraying loop, this project places heavy emphasis on reliability: the sensor is given time to stabilize on startup, every single Modbus response is cryptographically checksummed and structurally validated before its values are trusted, and the pump is completely electrically isolated from the Arduino through a relay, protecting the low-power microcontroller from the higher current and voltage spikes associated with driving a motor. The result is a practical, safe, and scalable proof-of-concept for precision agriculture that can be built with widely available components and modest cost.

## 2. Objective

The overarching goal of this project is to demonstrate that low-cost, widely available embedded hardware can replace manual, guesswork-based fertilization with an automatic, data-driven process. This objective is broken down into the following specific technical and practical goals:

### 2.1 Accurate Real-Time Nutrient Measurement

Continuously and accurately measure real soil Nitrogen, Phosphorus, and Potassium levels using an industrial-grade RS485 Modbus sensor, rather than relying on cheap analog sensors that cannot distinguish between the three nutrients, or on manual soil testing kits that require lab time and human interpretation.

### 2.2 Transparent, Live Feedback

Display live nutrient readings and system status on an OLED screen at all times, so that a farmer, student, or competition judge can see exactly what the soil condition is and what the system is doing at any given

moment - whether it is calibrating, reading, spraying, or reporting an error - without needing a laptop or Serial Monitor connected.

### 2.3 Threshold-Based Automatic Action

Automatically spray fertilizer only when a nutrient is measurably below a defined healthy threshold, using a simple but effective logical rule (spray if any one of N, P, or K is deficient), instead of a timer-based or manually-triggered spray, which wastes fertilizer and does not respond to actual soil conditions.

### 2.4 Data Integrity and Fault Tolerance

Prevent false or accidental triggers caused by corrupted, incomplete, or misdirected sensor data by validating every single Modbus response using CRC-16 checksums, slave ID matching, function code checking, and byte-count verification, and by ensuring the pump remains off whenever the sensor cannot be trusted.

### 2.5 Electrical Safety Through Isolation

Keep the fertilizer pump completely electrically isolated from the Arduino's logic circuitry by driving it through a relay module with its own independent power supply, protecting the microcontroller from motor back-EMF, current spikes, and voltage sag that could otherwise damage or reset the board.

## 3. Components Used

Every component in this project was chosen specifically for its role in either sensing, processing, displaying, or actuating the fertilization decision. The table below summarizes each component and its function, followed by a brief explanation of why each was necessary rather than a simpler alternative.

| Component | Purpose |
|---|---|
| Arduino UNO | Main microcontroller running the control logic, Modbus protocol, and display driving |
| RS485 Modbus RTU NPK Soil Sensor | Measures real Nitrogen, Phosphorus, and Potassium levels in soil |
| MAX485 TTL-to-RS485 Module | Converts Arduino's TTL serial to the RS485 differential signal the sensor needs |
| SSD1306 128x32 I2C OLED Display | Shows live NPK readings and system status text |
| 1-Channel Relay Module | Electrically isolated switch that turns the pump on and off |
| Low-Voltage DC Fertilizer Pump | Physically sprays the fertilizer solution when triggered |
| Separate Pump Power Supply | Powers the pump independently of the Arduino's own supply |

### Why an RS485 Modbus sensor instead of a cheap analog sensor?

Cheap capacitive or resistive soil sensors typically output a single analog voltage related to moisture, not nutrient content, and cannot distinguish between Nitrogen, Phosphorus, and Potassium at all. The RS485 Modbus sensor used here is a purpose-built agricultural instrument with internal electrochemical sensing elements and its own onboard microcontroller, which reports three separate, calibrated numeric values over a robust industrial communication standard - the same standard used in real greenhouse and farm automation systems.

### Why RS485 requires a MAX485 module

RS485 is a differential signaling standard designed for long-distance, noise-resistant communication, using two wires (A and B) rather than a single-ended TTL signal. The Arduino UNO's built-in UART only speaks TTL serial, so the MAX485 module acts as a translator in both directions, and also includes a direction-control pin (DE/RE) that the Arduino must toggle to switch between transmitting a request and receiving the sensor's reply, since RS485 is a half-duplex bus.

### Why a relay instead of driving the pump directly from a pin

An Arduino UNO's digital pins can safely supply only around 20-40 milliamps each, while even a small DC pump draws far more current, and generates a voltage spike (back-EMF) when it switches off. Driving it directly from a pin would risk instantly damaging the microcontroller. The relay module acts as a mechanically and electrically isolated switch: the Arduino only ever sends a small control signal to the relay's coil, while the pump's actual power circuit is completely separate.

## 4. System Wiring

The system is built from four physically and electrically distinct wiring groups: the sensor-to-MAX485 link, the MAX485-to-Arduino link, the OLED display, and the relay-to-pump circuit. Keeping these groups conceptually separate makes the system much easier to debug, since a fault in one group (for example, the pump) cannot electrically affect another (for example, the sensor communication).

### 4.1 MAX485 to Arduino UNO

| Arduino UNO | MAX485 Module |
|---|---|
| D2 | RO |
| D3 | DI |
| D4 | RE + DE (tied together) |
| 5V | VCC |
| GND | GND |

### 4.2 NPK Sensor to MAX485

| NPK Sensor | Connects To |
|---|---|
| A+ / RS485 A | MAX485 A |
| B- / RS485 B | MAX485 B |
| VCC / Power + | External supply matching sensor's rated voltage |
| GND / Power - | External supply GND (shared with Arduino/MAX485 GND) |

### 4.3 OLED Display (SSD1306, I2C, Address 0x3C)

| OLED | Arduino UNO |
|---|---|
| SDA | A4 |
| SCL | A5 |
| VCC | 5V |
| GND | GND |

### 4.4 Relay and Pump

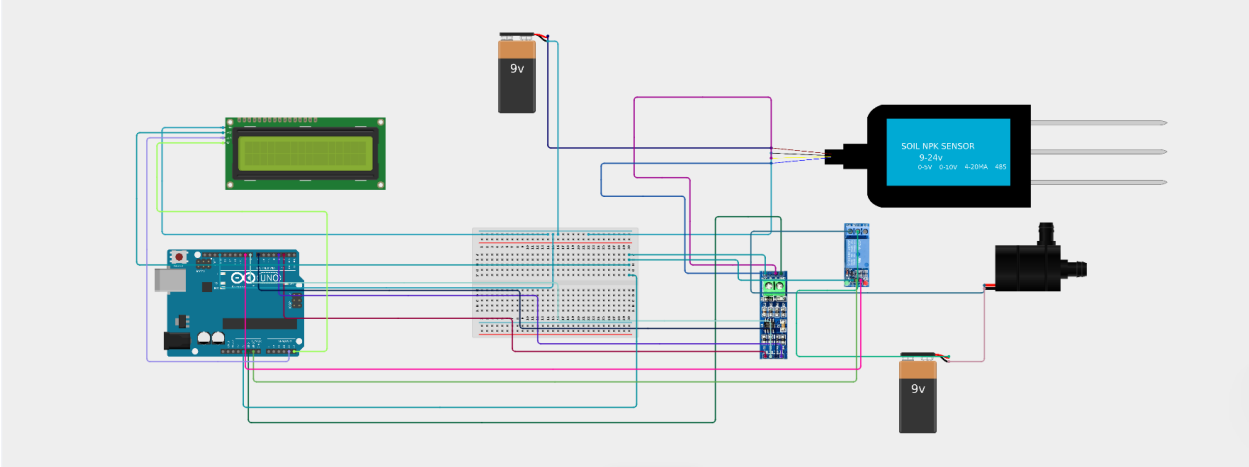
| Connection | Details |
|---|---|
| Arduino D7 -> Relay IN / I-P | Digital control signal only |
| Relay VCC -> Arduino 5V | Powers the relay's control circuit |
| Relay GND -> Arduino GND | Common ground for control side |
| Pump Power Supply (+) -> Relay COM | Separate supply matching pump's voltage |
| Relay NO -> Pump (+) | Switched output, closes only when relay is triggered |
| Pump (-) -> Pump Power Supply (-) | Completes the isolated pump circuit |

**Important Safety Note:** The fertilizer pump is never wired directly to an Arduino GPIO pin. The relay provides full electrical isolation between the Arduino's low-power logic and the pump's separate power circuit, protecting the microcontroller from voltage spikes and excess current draw. It is also strongly recommended to place a flyback (snubber) diode, such as a 1N4007, directly across the pump's two terminals to absorb the

voltage spike generated when the pump's motor switches off, which further protects the relay contacts from arcing or welding shut over repeated use.

## 4.5 Complete Circuit Diagram

The diagram below shows the complete assembled circuit, illustrating how the Arduino UNO, MAX485 module, NPK sensor, 16x2 LCD, relay module, and pump power supplies are all interconnected.



# 5. Working Principle

The system operates as a continuous, self-repeating control loop once powered on. Each stage of this loop is described in detail below, reflecting exactly how the logic is implemented in the Arduino sketch.

## Step 1: Startup Calibration (One-Time, 60 Seconds)

On power-up, before any reading is trusted, the system holds the pump firmly OFF and displays a live 60-second countdown on the OLED. This is necessary because electrochemical soil sensors, like this NPK sensor, need a short warm-up period after power-on for their internal sensing elements to stabilize; readings taken immediately at power-on are frequently inaccurate or read as zero.

## Step 2: Modbus Request Transmission

The Arduino constructs a Modbus RTU request frame consisting of the sensor's slave address, the function code 0x03 (Read Holding Registers), the starting register address for the N/P/K values, and the number of registers to read. A CRC-16 checksum is calculated over this frame and appended. The MAX485's direction pin is set to transmit mode, the frame is sent, and the module is immediately switched back to receive mode to listen for the sensor's reply.

## Step 3: Response Reception and Validation

The Arduino listens for up to 500 milliseconds for a complete response. Once received, it is passed through several independent checks (detailed fully in Section 8) before any value inside it is trusted. If any check fails, the reading is discarded entirely.

## Step 4: Threshold Comparison

If the response is fully valid, the three 16-bit register values are decoded into Nitrogen, Phosphorus, and Potassium readings in mg/kg. Each is compared against its configured threshold. If any single nutrient is below its threshold, the soil is classified as deficient for that cycle.

## Step 5: Automatic Spraying

If the soil is deficient, the OLED immediately shows 'PUMP ON: Spraying' along with the current readings, the relay is energized for exactly 3 seconds to run the pump, and then de-energized. The OLED then shows a live 20-second 'Checking results' countdown, giving the freshly applied fertilizer time to begin settling into the soil before the next reading is taken - this avoids immediately re-triggering another spray based on a reading taken before the fertilizer has had any effect.

## Step 6: Idle / Healthy State

If all three nutrients are sufficient, the pump stays off, the OLED shows the live readings with an 'OK: No spray needed' message, and the system waits a short 5-second interval before taking the next reading, keeping the displayed values continuously up to date.

## Step 7: Error Handling

If the sensor reading fails validation for any reason, the relay is explicitly forced off (regardless of its previous state), the OLED displays 'SENSOR ERROR / CHECK RS485', and the system waits before retrying. No previous, cached, or fabricated value is ever displayed or acted on in place of a genuine new reading.

This entire read - validate - compare - act sequence repeats indefinitely for as long as the system is powered, automatically adapting to changing soil conditions without any manual intervention.

## 6. Nutrient Thresholds

Nitrogen, Phosphorus, and Potassium are the three primary macronutrients required for healthy plant growth, and each plays a distinct physiological role. Understanding these roles explains why the system checks all three independently rather than relying on a single combined 'soil health' number.

| Nutrient | Role in Plant Growth | Threshold | Deficiency Symptoms |
|---|---|---|---|
| Nitrogen (N) | Drives leaf and stem growth; a core component of chlorophyll | 30 mg/kg | Yellowing leaves, stunted growth |
| Phosphorus (P) | Supports root development, flowering, and energy transfer | 20 mg/kg | Poor root growth, delayed flowering |
| Potassium (K) | Regulates water uptake and improves disease resistance | 150 mg/kg | Weak stems, scorched leaf edges |

The system sprays if **any one** of the three nutrients falls below its threshold - a logical OR condition - since a plant can suffer a serious deficiency in a single nutrient even while the other two remain fully sufficient. For example, a soil sample with abundant Nitrogen and Phosphorus but low Potassium would still leave a plant vulnerable to disease and water-stress, so the system correctly triggers spraying in that case rather than only reacting to an average or combined score.

These specific threshold values (30, 20, and 150 mg/kg) were selected as reasonable general-purpose defaults for demonstration purposes. In a real deployment, these numbers would be tuned to the specific crop being grown, since different plants have different optimal nutrient ranges - for example, leafy vegetables typically demand higher Nitrogen levels than root vegetables or flowering ornamentals. Because these values are implemented as named constants in the source code, they can be changed in seconds without altering any other part of the system's logic.

# 7. Timing Summary

Every timing value in this system was deliberately chosen based on the physical behavior of the sensor, the pump, and the soil - not arbitrarily. The table below summarizes each timing constant used in the system, followed by the reasoning behind each choice.

| Event | Duration |
|---|---|
| Startup sensor calibration (one-time) | 60 seconds |
| Spray duration per trigger | 3 seconds |
| Wait after spraying before rechecking | 20 seconds (live countdown shown) |
| Normal check interval (when nutrients OK) | 5 seconds |
| Modbus response timeout | 500 milliseconds |

### Why 60 seconds for startup calibration?

Electrochemical sensing elements inside the NPK probe require a short period after power-on to reach a stable electrical baseline. Testing showed that readings taken in the first several seconds after power-on were unreliable or read as zero; a 60-second warm-up window was found to be a practical balance between reliability and not making the operator wait excessively long before the system becomes active.

### Why a 3-second spray duration?

The spray duration determines how much fertilizer solution is delivered per trigger. Three seconds was chosen to deliver a small, controlled dose rather than flooding the soil, since the system is designed to spray repeatedly in small increments and recheck, rather than delivering one large dose that cannot be adjusted once applied.

### Why a 20-second post-spray recheck delay?

Immediately after spraying, the soil and sensor probe need a short window for the fertilizer solution to begin diffusing and for the sensor's readings to reflect the change. Rechecking too quickly would likely show almost no change and could cause the system to spray again unnecessarily in rapid succession. A 20-second pause, shown as a live countdown so the operator knows the system has not frozen, provides a more meaningful window before the next evaluation.

### Why a 500ms Modbus timeout?

At a baud rate of 9600, the sensor's 11-byte response transmits in only a few milliseconds; 500 milliseconds provides generous margin for the sensor's own internal processing delay while still keeping the system responsive and quickly able to detect a genuinely non-responding sensor rather than hanging indefinitely.

# 8. Data Validation (Reliability Safeguards)

A central engineering focus of this project is guaranteeing that the pump never sprays based on bad, corrupted, or fabricated data. RS485 lines, while generally robust, can still be affected by electrical noise, loose connections, or timing issues, so every single Modbus response is passed through several independent validation layers before any value inside it is trusted for a spraying decision.

| Check | What It Detects |
|---|---|
| Response Timeout (500ms) | Sensor did not reply at all (wiring, power, or address issue) |
| Modbus Exception Check | Sensor understood the request but rejected it (e.g. wrong register) |
| Slave ID Validation | The response did not actually come from the expected sensor address |
| Function Code Validation | The response type does not match the request that was sent |
| Byte Count Validation | The response contains the wrong number of data bytes |
| CRC-16 Validation | The data was corrupted or altered in transit on the RS485 line |

## How the CRC-16 check works

Modbus RTU uses a specific CRC-16 algorithm to detect transmission errors. Both the Arduino's outgoing request and the sensor's incoming response include a 2-byte checksum calculated over the rest of the message. Upon receiving a response, the Arduino independently recalculates the checksum over the received data bytes and compares it against the checksum bytes actually received. If even a single bit in the message was flipped or corrupted during transmission - due to electrical noise, a loose wire, or interference - the recalculated checksum will not match, and the entire response is rejected outright.

## What happens when validation fails

If any single one of these checks fails, the system does not spray, does not display a fabricated or estimated value, and critically, does not reuse a previous successful reading as if it were a new, current one. Instead, it clearly reports a 'SENSOR ERROR' state on the OLED and forces the relay off, ensuring the fertilizer pump can never be triggered by unreliable data.

# 9. Key Features

## 9.1 Genuine Sensor Data Only

Every nutrient value displayed or acted upon comes directly from the physical RS485 sensor via a fully validated Modbus transaction. The code contains no use of random number generation, no simulated or placeholder values, and no hardcoded readings anywhere - a design constraint enforced deliberately throughout development to ensure the system is a genuine sensing solution, not a demonstration using fake data.

## 9.2 Fully Automatic Operation

Once powered on, the system requires no manual switches, buttons, or scheduled timers of any kind. The entire read-validate-compare-act cycle runs continuously and indefinitely on its own, making it suitable for genuinely unattended, real-world deployment.

## 9.3 Continuous Live Feedback

The OLED display is updated at every distinct stage of operation - calibrating, checking results, actively spraying, healthy/idle, or reporting a sensor error - each with its own clear message and, where a wait is involved, a live countdown timer. This ensures the system never appears frozen or unresponsive to an observer, which was a specific design requirement addressed during development.

## 9.4 Electrical Isolation and Safety

The relay module provides complete electrical separation between the Arduino's sensitive low-voltage logic circuitry and the pump's separate, higher-current power circuit. This protects the microcontroller from motor back-EMF, current spikes, and possible voltage sag or reset events that could otherwise occur if the pump were driven directly from a GPIO pin.

## 9.5 Configurable, Adaptable Design

Every operationally significant value in the system - the Modbus slave ID, baud rate, register address, nutrient thresholds, spray duration, and all timing intervals - is implemented as a single named constant at the top of the code. This means the system can be adapted to a different sensor model, a different crop's nutrient requirements, or a different desired spray behavior without modifying any of the underlying control logic.

## 9.6 Robust Communication Protocol

Rather than relying on a simple analog signal, the system implements the genuine industrial Modbus RTU protocol in full, including manual RS485 direction control, correct CRC-16 byte ordering, and comprehensive response validation - the same fundamental protocol used in real industrial and agricultural automation equipment.

# 10. Applications

## 10.1 Precision Agriculture for Small Farms

Small and mid-size farms often cannot justify the cost of large-scale commercial precision agriculture systems. A low-cost Arduino-based system like this one can bring the same core benefit - fertilizing based on actual measured need rather than a fixed schedule - within reach of smaller operations, reducing fertilizer costs and improving crop consistency.

## 10.2 Greenhouses and Polyhouses

Controlled-environment growing operations already rely heavily on automation for irrigation, temperature, and humidity. This system extends that same automation philosophy to fertilization, allowing nutrient delivery to be managed with the same precision as other environmental factors already under automatic control.

## 10.3 Smart Home and Terrace Gardening

For hobbyist gardeners managing potted plants, terrace gardens, or small vegetable patches, this system removes the guesswork of manual fertilizing entirely, ensuring plants receive nutrients exactly when needed even if the owner is away or simply forgets a scheduled feeding.

## 10.4 Nursery and Plant-Health Monitoring

Commercial plant nurseries growing large numbers of seedlings or saplings can use this kind of system to maintain consistent, healthy nutrient levels across many plants without dedicating staff time to individual manual testing and feeding.

## 10.5 Educational and Competition Demonstration

This project also serves as a strong educational example of combining embedded systems, industrial communication protocols (Modbus RTU/RS485), real-world sensor integration, and safe actuator control - making it well suited for STEM demonstrations, robotics/IoT competitions, and AgriTech coursework.

# 11. Future Scope

## 11.1 Wireless Connectivity and Cloud Logging

Adding a WiFi (such as an ESP8266/ESP32 companion) or GSM module would allow nutrient readings and spray events to be logged to a cloud dashboard or mobile app in real time, enabling remote monitoring from anywhere and historical trend analysis over days, weeks, or growing seasons.

## 11.2 Additional Soil Sensors

Integrating soil moisture and pH sensors alongside the existing NPK sensor would provide a more complete picture of overall soil health, since nutrient availability to plants is also strongly affected by moisture content and soil pH, allowing even smarter, more holistic fertilization decisions.

## 11.3 On-Board Data Logging

Adding an SD card module and a real-time clock (RTC) would allow the system to record every nutrient reading and spray event with a timestamp locally, enabling long-term trend analysis of soil health even without any network connectivity.

## 11.4 Multi-Node Field Deployment

Because Modbus RTU natively supports multiple devices on a single RS485 bus, each distinguished by its slave ID, this system's architecture could be extended to support several NPK sensor nodes distributed across a larger field, all communicating with a single central Arduino or upgraded controller.

## 11.5 Solar-Powered Off-Grid Operation

Pairing the system with a small solar panel and battery bank would allow it to operate completely off-grid, making it viable for remote agricultural plots without reliable access to mains electricity.

## 11.6 Adaptive/Machine-Learning-Based Thresholds

With sufficient logged historical data, future versions could move beyond fixed thresholds toward adaptive models that adjust nutrient targets based on the specific crop's growth stage, seasonal patterns, or observed plant response to previous fertilization events.

# 12. Conclusion

This project successfully demonstrates that a complete, reliable, and safety-conscious automatic fertilization system can be built entirely on an accessible, low-cost Arduino UNO platform. By combining genuine industrial sensor communication over RS485 Modbus RTU, rigorous multi-layer data validation including CRC-16 checksumming, clear and continuous real-time feedback through an OLED display, and safely isolated actuator control through a relay module, the system directly addresses a real and costly agricultural problem: applying fertilizer in the right amount, at the right time, based on the soil's actual, measured condition rather than a fixed schedule or human guesswork.

The system's emphasis on data integrity - refusing to act on unvalidated, corrupted, or stale sensor data - distinguishes it from simpler automation projects that assume sensor readings are always correct. Combined with its fully configurable design, in which every threshold and timing value is a single, clearly-named constant, the system is not just a fixed demonstration but a genuinely adaptable foundation that could be tuned for different crops, different sensors, or different environmental conditions with minimal rework.

Ultimately, this project shows that meaningful steps toward precision agriculture do not require expensive, proprietary commercial systems. With careful attention to protocol correctness, data validation, and electrical safety, a student-built embedded system using widely available components can deliver a genuinely practical, scalable, and educational solution to real-world fertilizer waste and crop health management.

## Appendix A: Complete Arduino Source Code

The full Arduino UNO sketch implementing the sensor communication, data validation, threshold logic, OLED display, and pump control described throughout this report is listed below in its entirety.

```
/*
  ============================================================
  Automatic NPK-Based Fertilizer Spraying System
  Arduino UNO
  ============================================================

  Components:
    - Arduino UNO
    - RS485 Modbus RTU NPK soil sensor
    - MAX485 TTL-to-RS485 module
    - SSD1306 128x32 I2C OLED display (0x3C)
    - 1-channel relay module
    - Low-voltage DC fertilizer pump (powered separately, NOT from Arduino pin)

  REAL sensor readings only. No simulated/random/hardcoded NPK values.

  ---------------------- WIRING ----------------------
  MAX485 -> Arduino UNO:
    RO  -> D2
    DI  -> D3
    RE  -> D4
    DE  -> D4   (RE and DE tied together)
    VCC -> 5V
    GND -> GND

  NPK Sensor -> MAX485:
    A+ -> MAX485 A
    B- -> MAX485 B
    Sensor VCC -> external supply matching sensor's rated voltage
    Sensor GND -> external supply GND (shared with Arduino/MAX485 GND)

  OLED (SSD1306 128x32, I2C) -> Arduino UNO:
    SDA -> A4
    SCL -> A5
    VCC -> 5V
    GND -> GND
    Address -> 0x3C

  Relay -> Arduino UNO:
    IN  -> D7
    VCC -> 5V (or per relay module spec)
    GND -> GND

  Pump:
    Pump is switched by the relay's COM/NO contacts using its OWN
    separate power supply. Pump is NEVER wired directly to an
    Arduino GPIO pin. D7 only drives the relay's control input.

  ---------------------- LIBRARIES ----------------------
  Wire.h              (built-in)
  SoftwareSerial.h    (built-in)
  Adafruit_GFX.h      (install "Adafruit GFX Library")
  Adafruit_SSD1306.h  (install "Adafruit SSD1306")

  ============================================================
  MODBUS SETTINGS BELOW ARE COMMON DEFAULTS FOR THESE SENSORS,
  NOT GUARANTEED FOR YOUR EXACT UNIT. Check your sensor's
  manual/sticker and adjust if readings fail.
  ============================================================
*/

#include <SoftwareSerial.h>
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>

// ================ CONFIGURABLE MODBUS SETTINGS ================
#define MODBUS_SLAVE_ID   0x01     // Suspected default slave ID
#define BAUD_RATE         9600     // Suspected default baud rate
#define NPK_START_REG     0x001E   // Suspected register address for N (30 decimal)
#define NPK_REG_COUNT     3        // N, P, K = 3 registers
// ==============================================================

// ================ NUTRIENT THRESHOLDS ================
#define N_THRESHOLD   30    // mg/kg
#define P_THRESHOLD   20    // mg/kg
#define K_THRESHOLD   150   // mg/kg
// =====================================================
#define MAX485_RO_PIN   2   // Arduino RX <- MAX485 RO
```

```
#define MAX485_DI_PIN   3   // Arduino TX -> MAX485 DI
#define MAX485_DE_RE    4   // Shared direction control pin

#define RELAY_PIN       7   // Controls relay, NOT the pump directly

#define RESPONSE_TIMEOUT_MS   500
#define CHECK_INTERVAL_MS     5000  // Normal check interval when nutrients are fine
#define SPRAY_DURATION_MS     3000  // Pump runs for exactly 3 seconds
#define CALIBRATION_TIME_MS   60000 // One-time sensor warm-up on startup (1 min)
#define POST_SPRAY_WAIT_MS    20000 // Wait after spraying before rechecking (20 sec)

// ---------------- OLED settings ----------------
#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 32
#define OLED_RESET    -1
#define OLED_ADDRESS  0x3C

Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, OLED_RESET);
SoftwareSerial rs485(MAX485_RO_PIN, MAX485_DI_PIN); // RX, TX

// ---------------- Modbus CRC-16 calculation ----------------
uint16_t modbusCRC16(uint8_t *buf, uint8_t len) {
  uint16_t crc = 0xFFFF;
  for (uint8_t pos = 0; pos < len; pos++) {
    crc ^= (uint16_t)buf[pos];
    for (uint8_t i = 8; i != 0; i--) {
      if (crc & 0x0001) {
        crc >>= 1;
        crc ^= 0xA001;
      } else {
        crc >>= 1;
      }
    }
  }
  return crc;
}

// ---------------- RS485 direction control ----------------
void rs485Transmit() {
  digitalWrite(MAX485_DE_RE, HIGH);
  delay(2);
}

void rs485Receive() {
  digitalWrite(MAX485_DE_RE, LOW);
  delay(2);
}

// ---------------- Send Modbus request & read response ----------------
bool readNPK(uint16_t &nitrogen, uint16_t &phosphorus, uint16_t &potassium) {
  uint8_t request[8];
  request[0] = MODBUS_SLAVE_ID;
  request[1] = 0x03; // Function code: Read Holding Registers
  request[2] = (NPK_START_REG >> 8) & 0xFF;
  request[3] = NPK_START_REG & 0xFF;
  request[4] = (NPK_REG_COUNT >> 8) & 0xFF;
  request[5] = NPK_REG_COUNT & 0xFF;

  uint16_t crc = modbusCRC16(request, 6);
  request[6] = crc & 0xFF;        // CRC low byte first
  request[7] = (crc >> 8) & 0xFF; // CRC high byte second

  while (rs485.available()) rs485.read(); // flush stale data

  rs485Transmit();
  rs485.write(request, 8);
  rs485.flush();
  rs485Receive();

  const uint8_t EXPECTED_LEN = 11; // slaveID+func+byteCount+6data+2CRC
  uint8_t response[EXPECTED_LEN];
  uint8_t receivedLen = 0;

  unsigned long startTime = millis();
  while ((millis() - startTime) < RESPONSE_TIMEOUT_MS && receivedLen < EXPECTED_LEN) {
    if (rs485.available()) {
      response[receivedLen] = rs485.read();
      receivedLen++;
    }
  }

  if (receivedLen == 0) {
    Serial.println(F("ERROR: No response from sensor (timeout)."));
    return false;
  }
  if (receivedLen >= 3 && (response[1] & 0x80)) {
    Serial.print(F("ERROR: Modbus exception. Code: 0x"));
    Serial.println(response[2], HEX);
```

```
    return false;
  }

  if (receivedLen < EXPECTED_LEN) {
    Serial.print(F("ERROR: Incomplete response. Bytes: "));
    Serial.println(receivedLen);
    return false;
  }

  if (response[0] != MODBUS_SLAVE_ID) {
    Serial.print(F("ERROR: Slave ID mismatch. Got: "));
    Serial.println(response[0]);
    return false;
  }

  if (response[1] != 0x03) {
    Serial.print(F("ERROR: Unexpected function code: 0x"));
    Serial.println(response[1], HEX);
    return false;
  }

  uint8_t byteCount = response[2];
  if (byteCount != (NPK_REG_COUNT * 2)) {
    Serial.print(F("ERROR: Unexpected byte count: "));
    Serial.println(byteCount);
    return false;
  }

  uint16_t receivedCRC = response[9] | (response[10] << 8);
  uint16_t calculatedCRC = modbusCRC16(response, 9);
  if (receivedCRC != calculatedCRC) {
    Serial.print(F("ERROR: CRC mismatch. Recv: 0x"));
    Serial.print(receivedCRC, HEX);
    Serial.print(F(" Calc: 0x"));
    Serial.println(calculatedCRC, HEX);
    return false;
  }

  nitrogen   = (response[3] << 8) | response[4];
  phosphorus = (response[5] << 8) | response[6];
  potassium  = (response[7] << 8) | response[8];

  return true;
}

// ---------------- OLED display helpers ----------------
void showReadingOnOLED(uint16_t n, uint16_t p, uint16_t k, bool pumpOn) {
  display.clearDisplay();
  display.setTextSize(1);
  display.setCursor(0, 0);
  display.print(F("N:"));
  display.print(n);
  display.print(F(" P:"));
  display.print(p);
  display.print(F(" K:"));
  display.println(k);

  display.setCursor(0, 16);
  if (pumpOn) {
    display.println(F("PUMP ON: Spraying"));
  } else {
    display.println(F("OK: No spray needed"));
  }

  display.display();
}

void showErrorOnOLED() {
  display.clearDisplay();
  display.setTextSize(1);
  display.setCursor(0, 0);
  display.println(F("SENSOR ERROR"));
  display.setCursor(0, 16);
  display.println(F("CHECK RS485"));
  display.display();
}

void showCheckingResultsCountdown(unsigned long durationMs) {
  unsigned long startTime = millis();
  unsigned long remainingSec = durationMs / 1000;

  // Draw the first frame immediately so it doesn't look frozen
  display.clearDisplay();
  display.setTextSize(1);
  display.setCursor(0, 0);
  display.println(F("Checking results"));
  display.setCursor(0, 16);
  display.print(F("Wait: "));
```

```
  display.print(remainingSec);
  display.println(F("s"));
  display.display();

  while (millis() - startTime < durationMs) {
    unsigned long elapsedSec = (millis() - startTime) / 1000;
    unsigned long secLeft = (durationMs / 1000) - elapsedSec;
    if (secLeft != remainingSec) {
      remainingSec = secLeft;
      display.clearDisplay();
      display.setCursor(0, 0);
      display.println(F("Checking results"));
      display.setCursor(0, 16);
      display.print(F("Wait: "));
      display.print(remainingSec);
      display.println(F("s"));
      display.display();
      Serial.print(F("Rechecking in... "));
      Serial.print(remainingSec);
      Serial.println(F("s"));
    }
  }
}

// ---------------- Pump control ----------------
void runPump() {
  digitalWrite(RELAY_PIN, HIGH); // relay ON -> pump ON
  Serial.println(F("Pump ON: spraying for 3 seconds."));
  delay(SPRAY_DURATION_MS);
  digitalWrite(RELAY_PIN, LOW);  // relay OFF -> pump OFF
  Serial.println(F("Pump OFF."));
}

// ---------------- Setup ----------------
void setup() {
  // Capture and print WHY the Arduino just started, before it's cleared.
  // This proves whether it was a real power-on, or an unwanted reset
  // caused by the pump/relay glitching the board.
  uint8_t resetReason = MCUSR;
  MCUSR = 0; // clear for next time

  Serial.begin(9600);
  Serial.print(F("Startup reset reason (MCUSR = 0x"));
  Serial.print(resetReason, HEX);
  Serial.println(F("):"));
  if (resetReason & (1 << PORF))  Serial.println(F("  -> Power-on reset (normal, expected on first plug-in)"));
  if (resetReason & (1 << EXTRF)) Serial.println(F("  -> External reset (reset button / DTR from USB)"));
  if (resetReason & (1 << BORF))  Serial.println(F("  -> Brown-out reset (VOLTAGE DIP - pump/relay likely glitching power!)"));
  if (resetReason & (1 << WDRF))  Serial.println(F("  -> Watchdog reset"));

  pinMode(MAX485_DE_RE, OUTPUT);
  rs485Receive();

  pinMode(RELAY_PIN, OUTPUT);
  digitalWrite(RELAY_PIN, LOW); // ensure pump OFF at startup

  rs485.begin(BAUD_RATE);

  if (!display.begin(SSD1306_SWITCHCAPVCC, OLED_ADDRESS)) {
    Serial.println(F("OLED NOT FOUND. Check wiring."));
    while (1); // halt
  }

  display.clearDisplay();
  display.setTextColor(SSD1306_WHITE);
  display.setTextSize(1);
  display.setCursor(0, 0);
  display.println(F("NPK Fertilizer"));
  display.println(F("System Init..."));
  display.display();
  delay(1500);
  display.clearDisplay();
  // ---------------- Sensor calibration / warm-up delay ----------------
  // Pump stays OFF the entire time. Readings taken right after power-up
  // are not stable, so we wait before trusting any NPK values.
  Serial.println(F("Calibrating NPK sensor, please wait..."));
  digitalWrite(RELAY_PIN, LOW); // pump stays OFF during calibration

  unsigned long calibrationStart = millis();
  unsigned long remainingSec = CALIBRATION_TIME_MS / 1000;

  while (millis() - calibrationStart < CALIBRATION_TIME_MS) {
    unsigned long elapsedSec = (millis() - calibrationStart) / 1000;
    unsigned long secLeft = (CALIBRATION_TIME_MS / 1000) - elapsedSec;
    if (secLeft != remainingSec) {
      remainingSec = secLeft;
      display.clearDisplay();
      display.setCursor(0, 0);
```

```
      display.println(F("Calibrating sensor"));
      display.print(F("Wait: "));
      display.print(remainingSec);
      display.println(F("s"));
      display.display();

      Serial.print(F("Calibrating... "));
      Serial.print(remainingSec);
      Serial.println(F("s remaining"));
    }
  }

  display.clearDisplay();
  display.setCursor(0, 0);
  display.println(F("Calibration done"));
  display.display();
  delay(1000);
  display.clearDisplay();

  Serial.println(F("NPK fertilizer spraying system started."));
}

// ---------------- Main loop ----------------
void loop() {
  uint16_t nitrogen = 0, phosphorus = 0, potassium = 0;

  bool success = readNPK(nitrogen, phosphorus, potassium);
  unsigned long waitTime = CHECK_INTERVAL_MS; // default wait before next check

  if (success) {
    Serial.print(F("N: "));
    Serial.print(nitrogen);
    Serial.print(F(" mg/kg   P: "));
    Serial.print(phosphorus);
    Serial.print(F(" mg/kg   K: "));
    Serial.print(potassium);
    Serial.println(F(" mg/kg"));
    bool nutrientLow = (nitrogen < N_THRESHOLD) ||
                       (phosphorus < P_THRESHOLD) ||
                       (potassium < K_THRESHOLD);

    if (nutrientLow) {
      showReadingOnOLED(nitrogen, phosphorus, potassium, true);
      runPump();

      // After spraying, show a live 20s countdown before taking the
      // next real reading, instead of the normal 5s interval.
      Serial.println(F("Waiting 20s before rechecking nutrient levels..."));
      showCheckingResultsCountdown(POST_SPRAY_WAIT_MS);
      waitTime = 0; // countdown already consumed the wait time
    } else {
      // Nutrients are fine - show the real-time values on OLED.
      Serial.println(F("Nutrients sufficient. No spraying needed."));
      showReadingOnOLED(nitrogen, phosphorus, potassium, false);
      waitTime = CHECK_INTERVAL_MS;
    }
  } else {
    Serial.println(F("Reading failed. Pump remains OFF. Displaying SENSOR ERROR."));
    digitalWrite(RELAY_PIN, LOW); // ensure pump stays OFF on sensor failure
    showErrorOnOLED();
    waitTime = CHECK_INTERVAL_MS;
  }

  delay(waitTime);
}
```